

Getting Started with Java: A Guided Foundation

Java is a general-purpose programming language used in many different settings. It appears in business software, Android applications, games, and many other programs. Its flexibility makes it useful both for learning programming fundamentals and for building professional software.

A major reason Java is widely used is that Java programs are designed to work across different operating systems through an intermediate format called [bytecode](#). Before exploring Java syntax, it helps to understand how programs move from human-written instructions to instructions a computer can execute.

Source Code, Machine Code, and Bytecode

Computers ultimately understand [machine code](#), which is represented in binary as combinations of ones and zeros. Machine code is very close to the hardware, so it is considered a [low-level language](#). Although computers can process it directly, it is difficult for humans to read and write.

Programmers instead write [source code](#), which uses words, symbols, and rules that humans can understand more easily. A Java source-code file uses the `.java` file extension.

A compiler translates source code into a form that can be executed. In many programming languages, compiling source code directly produces machine code specific to a particular operating system or processor. Code compiled on one type of computer may not run on another type of computer.

Java solves much of this portability problem by using [bytecode](#). The process is:

1. A programmer writes Java source code in a `.java` file.
2. The Java compiler converts the source code into bytecode.
3. The bytecode is stored in a `.class` file.
4. A Java Virtual Machine translates the bytecode into machine code for the particular computer.

Bytecode is therefore a cross-platform intermediate format. The same compiled Java program can be moved between operating systems, provided that the destination computer has a suitable Java Virtual Machine.

An IDE usually organizes code into projects. A project contains the source files and configuration information needed for a program. In Eclipse, a project can be created through the project explorer or a file-and-new-project menu.

Creating a Java Class

A [class](#) is a collection of related code. Classes are central to Java because Java programs are organized around them. More advanced Java programming uses classes to represent objects and their behavior.

When a class named `Main` is created, Java normally stores it in a file named `Main.java`. The class name and file name are expected to correspond when the class is public.

A basic Java class looks like this:

```
public class Main {  
  
}
```

The curly braces define the body of the class. Code placed between these braces belongs to the class.

The Main Method

A Java application begins execution in a special method called the [main method](#):

```
public static void main(String[] args) {  
  
}
```

Each part has a role:

- `public` allows the method to be accessed by the Java runtime.
- `static` allows the method to be called without first creating an object of the class.
- `void` means the method does not return a value.
- `main` is the required method name for the program's starting point.
- `String[] args` allows command-line arguments to be received.
- The curly braces contain the instructions executed by the method.

A complete minimal program is:

```
public class Main {  
    public static void main(String[] args) {  
  
    }  
}
```

When the program runs, Java looks for the main method and begins executing the instructions inside it from top to bottom. If the main method is missing, the runtime cannot find the program's starting point and reports an error.

"The main method is the entry point of a Java application. Execution begins at the first statement inside that method and proceeds downward."

Printing Output/Displaying output

Java can display information in the console by using `System.out`.

print

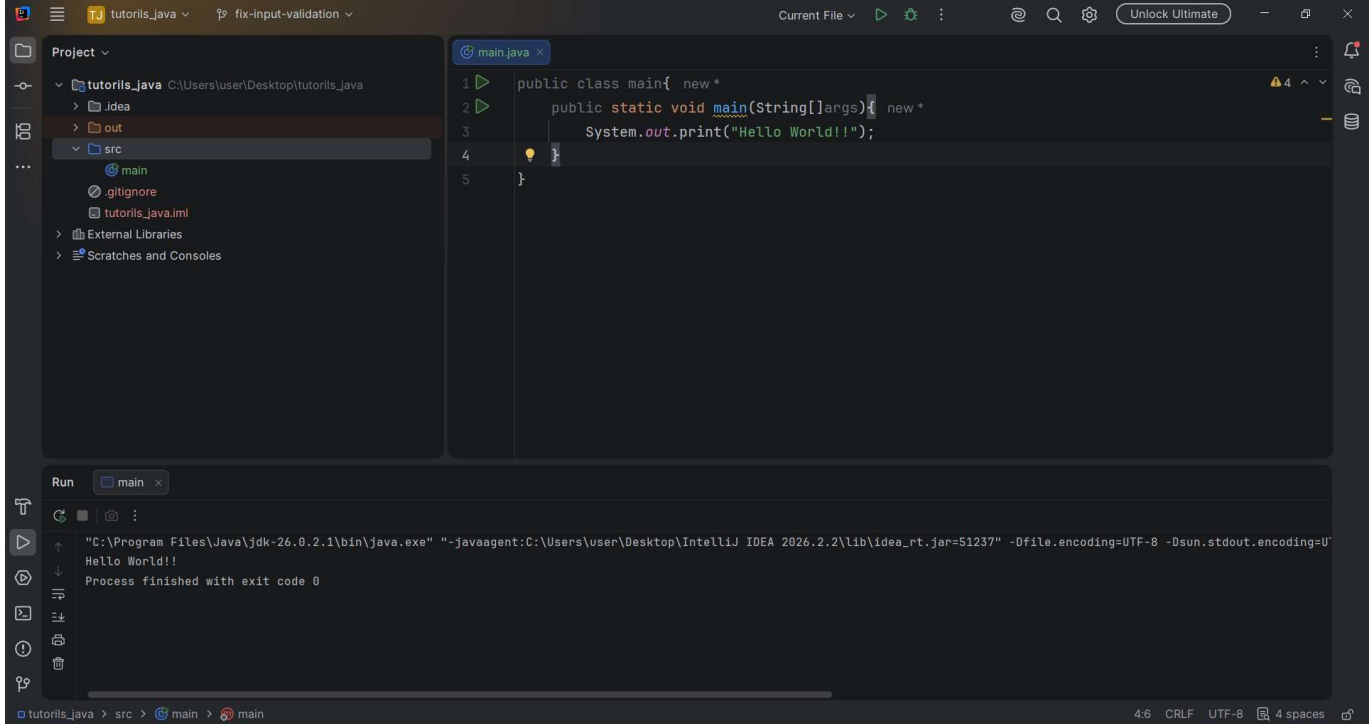
The `print` method displays text but does not automatically move the cursor to a new line.

```
System.out.print("Hello World!!");
```

The output appears on one line:

```
Hello world!!
```

Text placed inside double quotation marks is called a [string literal](#).



println

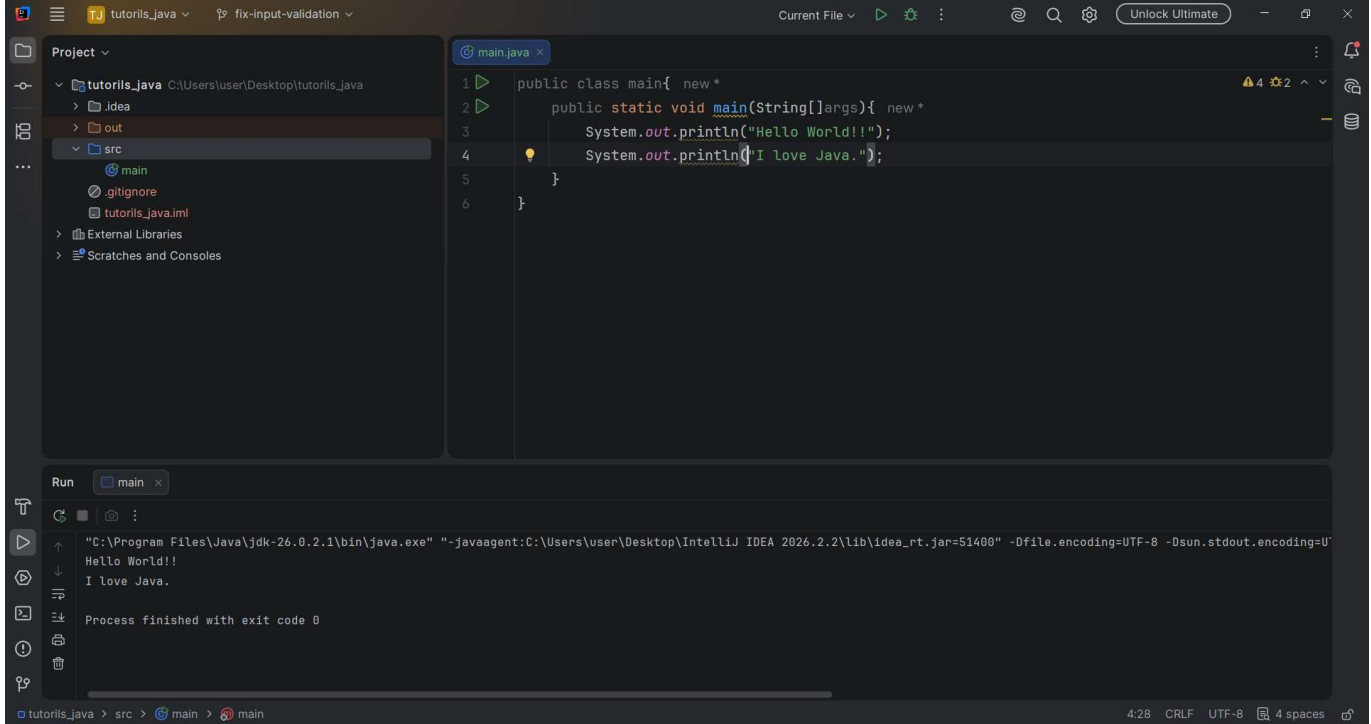
The `println` method displays text and then moves the cursor to the next line.

```
System.out.println("Hello World!!");  
System.out.println("I love Java.");
```

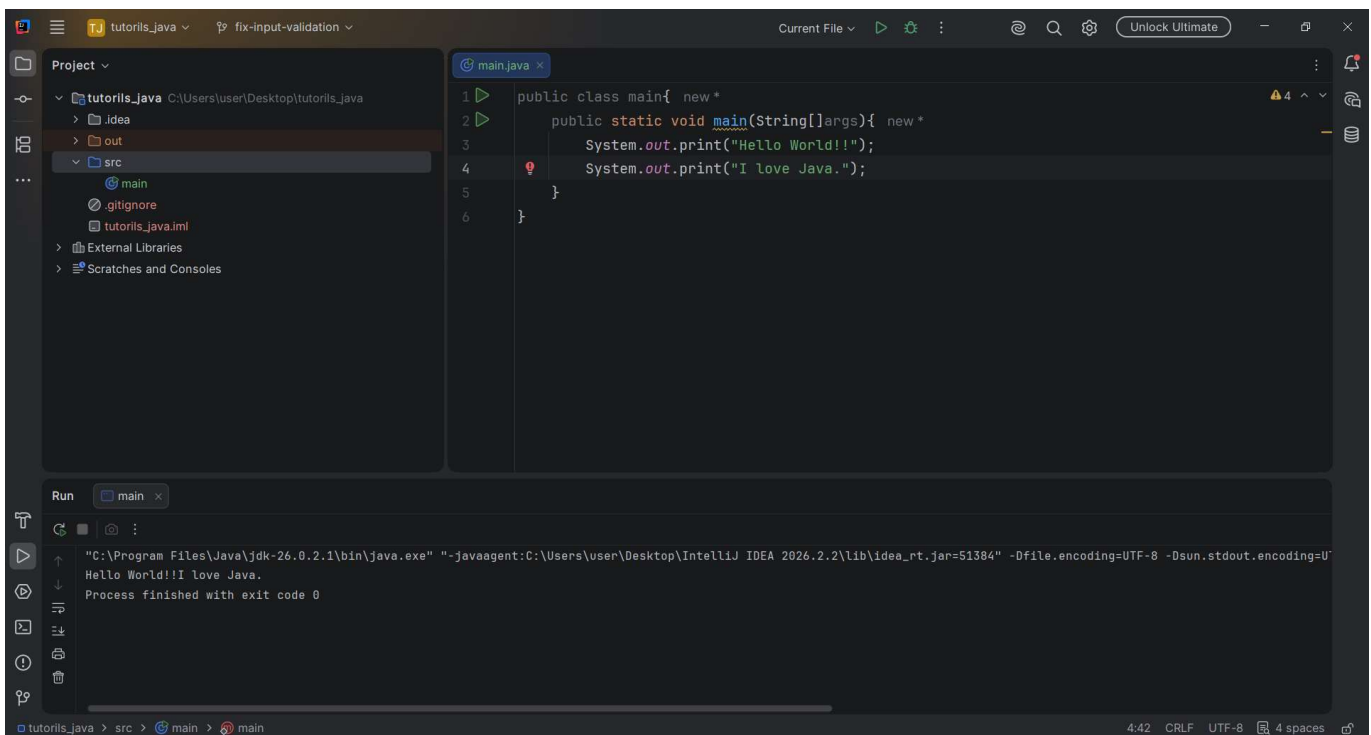
The output is:

```
Hello World!!  
I love Java.
```

The difference is that `println` behaves as though the Enter key were pressed after the text is printed.



Without In added to the println



Escape Sequences

An [escape sequence](#) begins with a backslash and tells Java to interpret the following character specially. Escape sequences allow a string to contain formatting or characters that would otherwise be difficult to represent.

Comments

A [comment](#) is text written for human readers that the compiler ignores. Comments can explain code, leave reminders, or make a program easier for another programmer to understand.

A single-line comment begins with two forward slashes:

```
// This line explains the next instruction.  
System.out.println("Hello");
```

Everything after `//` on that line is ignored by the compiler.

A multi-line comment begins with `/*` and ends with `*/`:

```
/*  
    This is a multi-line comment.  
    The compiler ignores all of it.  
*/  
System.out.println("Hello");
```

Comments do not change the behavior of a program. Their purpose is communication and documentation.